

Relatório de Auditoria de Segurança

Floresca Floricultura

Data: 11/09/2026

Escopo: Código-fonte completo (server actions, API routes, auth, frontend, infra)

Stack Tecnológica Detectada

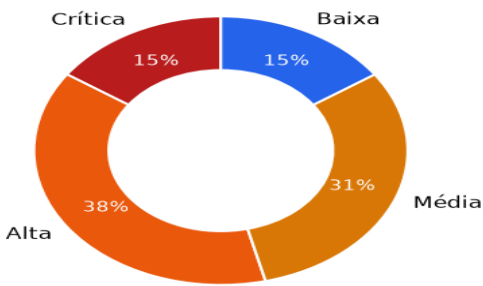
Componente	Tecnologia
Framework	Next.js 16 (App Router, Turbopack)
Linguagem	TypeScript
ORM / DB	Prisma Composer + raw SQL (pg driver) / Neon PostgreSQL
Autenticação	NextAuth v5 (JWT strategy, Google OAuth + Credentials)
Frontend	React, Tailwind CSS, Mercado Pago Card Payment Brick
Pagamentos	Mercado Pago SDK v3 (Checkout Transparente + Preference)
Email	Resend (React Email)
Deploy	Vercel (previsto)

1. Resumo Executivo

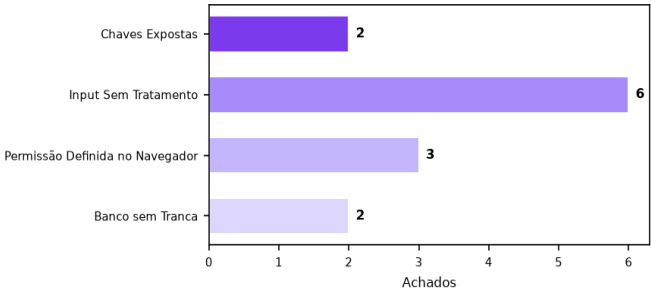
Esta auditoria identificou **13 achados** de segurança no código-fonte do projeto Floresca Floricultura:

[Crítica] 2 achado(s) **[Alta]** 5 achado(s) **[Média]** 4 achado(s) **[Baixa]** 2 achado(s)

Distribuição por Severidade



Achados por Categoria



2. Pontos Fortes Verificados

Item	Descrição
Autenticação JWT	NextAuth v5 com JWT strategy configurada corretamente. Sessões são stateless e seguras.
Middleware de Auth	middleware.ts protege corretamente todas as rotas /admin com verificação de login e role.
requireAdmin()	Server actions de escrita (create, update, delete) chamam requireAdmin() corretamente.
Validação de Input	CheckoutSchema e outros schemas Zod validam todos os campos de entrada.
Webhooks Route	A rota webhook valida o tipo de notificação e o paymentId antes de processar.
.gitignore	O arquivo .env está excluído do versionamento (verificado via git ls-files).
SQL Parametrizado	As queries em auth.ts usam parâmetros \$1, \$2 corretamente (sem injeção SQL).

3. Achados Detalhados

Severidade	Arquivo:Linha	Descrição
Crítica	payments.ts:163-170	Token de cartão de crédito logado no console O token de pagamento (cardData.token) é impresso via console.log no servidor. Viola PCI-DSS e pode expor dados sensíveis em sistemas de log.
Crítica	payments.ts:183-184	CPF hardcoded em pagamentos com cartão O CPF '12345678909' é enviado para TODOS os pagamentos com cartão. O CPF real do cliente nunca é coletado no formulário de checkout. Risco de fraude e falha KYC.
Alta	webhooks/mercadopago/route.ts:10-16	Assinatura de webhook nunca verificada A função verifyWebhookSignature existe mas NUNCA é chamada. Um atacante pode forjar requisições para marcar qualquer Venda como 'aprovado'.
Alta	payments.ts:47-48	Race condition na geração de IDs Lê todas as linhas para calcular MAX(id)+1. Duas requisições concorrentes podem gerar o mesmo nextId, causando colisão de chave primária.
Alta	vendas.ts:12-28, products.ts:13-16	Server actions de dados expostas sem autenticação getVendas(), getProducts(), getFuncionarios(), getClientes() não chamam requireAdmin(). Qualquer visitante pode invocar essas server actions e acessar dados.
Alta	products.ts:85-97	Upload de imagem sem validação server-side Não há validação de tipo de arquivo (magic bytes), limite de tamanho, ou restrição de extensão no servidor. Um arquivo .html ou .svg com extensão .avif seria servido como estático.
Alta	auth.ts:54-75	Login sem rate limiting ou bloqueio de conta O provider credentials não possui limite de tentativas de login, bloqueio de conta após falhas, ou proteção contra brute force.
Média	funcionarios.ts:13	SQL injection potencial via string interpolation A função getNextId usa template string com table e idColumn. Hoje são valores hardcoded, mas o padrão é perigoso para manutenção futura.
Média	checkout/page.tsx:27	JSON.parse sem tratamento de erro localStorage.getItem() + JSON.parse() sem try/catch. Dados corrompidos causam crash do componente.
Média	payments.ts:88, 160	Geração de email sintético quando omitido Se o cliente não informa email, gera-se @floresca.com falso. Confirmações de pagamento vão para endereço inexistente.
Média	contract.prisma	Ausência de Row-Level Security no banco Nenhuma política RLS definida. Se DATABASE_URL vaza, todo o banco fica exposto. Não há isolamento de tenant/usuário no nível do banco.

Baixa	.env:4-18	Credenciais no arquivo .env (testes) O arquivo .env contém credenciais de teste (Mercado Pago, Google OAuth, Neon DB, Resend). Embora .gitignore o exclua, o arquivo está presente no filesystem.
Baixa	funcionario-table.tsx:58	CPF de funcionários exibido sem mascaramento CPFs são exibidos completos na tabela admin. Deveriam ser parcialmente mascarados (ex: ***.456.789-**).

4. Recomendações Priorizadas

Prioridade	Ações
P1 — Imediato (Crítica)	• Remover console.log de tokens de pagamento em payments.ts • Coletar CPF real do cliente no checkout e usar no pagamento
P2 — Alta (1-2 semanas)	• Ativar verificação de assinatura de webhook (verifyWebhookSignature) • Adicionar requireAdmin() em todas as server actions de leitura • Implementar rate limiting no login (max 5 tentativas/minuto) • Validar uploads de imagem no servidor (magic bytes, tamanho, extensão) • Corrigir race condition na geração de IDs (usar sequences do banco)
P3 — Média (1 mês)	• Refatorar unsafe() para parameterized queries • Adicionar try/catch no JSON.parse do checkout • Implementar RLS no banco de dados • Remover fallback de email sintético @floresca.com
P4 — Baixa (Backlog)	• Mascara CPFs na tabela de funcionários • Usar .env.example para documentar variáveis de ambiente